

Laboratorio 7

Información

Integrantes:

Name	Institution ID	GitHub User
Josué Say	22801	JosueSay
Carlos Valladares	221164	vgcarlol

- [Repositorio](#)

Preparación de entorno

```
In [ ]: # %pip install -r requirements.txt
        # jupyter nbconvert Lab7.ipynb --to html
```

DataSet: <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>

Task 1

El siguiente conjunto de preguntas evalúa su capacidad de analizar, justificar y argumentar decisiones técnicas de entrenamiento en un contexto médico real. No basta con mencionar fórmulas; se espera que usted conecte cada concepto con sus implicaciones prácticas dentro del proyecto MediScan Guatemala.

Pregunta 1.1

El médico coordinador del proyecto le presenta la siguiente situación: *"Tenemos 800 radiografías etiquetadas por nuestros radiólogos. Un colega me dijo que con tan pocos datos el modelo va a memorizar todo y no va a servir para nada en producción."*

Como ingeniero de IA a cargo, usted decide aplicar **Data Augmentation** como parte de la solución. Sin embargo, el médico le pregunta: *"¿No estamos inventando datos falsos que podrían confundir al modelo?"*

Con esto en mente, responda las siguientes preguntas en su reporte:

Inciso 1

Explique al médico, en términos que él pueda entender, qué es el Data Augmentation y por qué las imágenes generadas no son datos falsos. Use la analogía que considere más apropiada.

Respuesta:

Data Augmentation no “inventa” información médica nueva, sino que expone al modelo a variaciones realistas de las mismas radiografías que ya fueron validadas por los radiólogos.

Una forma intuitiva de verlo es pensar en un médico residente en formación. No aprende viendo una única radiografía perfectamente alineada, sino múltiples versiones del mismo caso: imágenes con distinta iluminación, ligeramente rotadas, con variaciones de contraste o tomadas en equipos distintos. Es el mismo paciente, la misma patología, pero en condiciones ligeramente diferentes. Eso no es información falsa; es **la misma realidad clínica observada bajo distintas condiciones**.

Data Augmentation hace exactamente eso: toma una radiografía real y genera variaciones (por ejemplo, pequeños cambios de orientación o brillo) para que el modelo no memorice una imagen específica, sino que aprenda el patrón subyacente (como las opacidades pulmonares). En lugar de “engañar” al modelo, lo que hace es **forzarlo a generalizar**, que es precisamente lo que necesitamos para producción en clínicas donde las condiciones de captura nunca son idénticas.

Inciso 2

En el contexto específico de radiografías de tórax, proponga tres transformaciones de Data Augmentation que serían válidas y justifique cada una. Luego, identifique una transformación que no debería aplicarse en este dominio médico y explique por qué podría comprometer la integridad diagnóstica del modelo.

Respuesta:

En radiografías de tórax, las transformaciones deben respetar la anatomía y la lógica clínica. No se trata de “aumentar datos” de cualquier forma, sino de simular variabilidad real del entorno hospitalario.

Transformaciones válidas:

1. Perturbaciones geométricas controladas

Desde el punto de vista del paciente donde puede estar ligeramente inclinado o el encuadre puede variar entre técnicos y centros. Por lo que introducir pequeñas rotaciones y recortes controlados no cambia la anatomía, pero sí rompe la rigidez espacial del dataset obteniendo como beneficio que el modelo deja de depender de una posición ideal de los pulmones y aprender a identificar patrones patológicos como

opacidades independientemente de pequeñas variaciones de adquisición, lo cual es clave para clínicas en entornos donde la estandarización llega a ser limitada.

2. Variabilidad radiométrica

Enfocado al punto de vista del equipo médico puede que se lleguen a tener en el entorno diferencias de calibración del equipo, exposición o digitalización que generan imágenes con distintos niveles de intensidad y contraste. Simular estas condiciones obliga al modelo a enfocarse en estructuras relevantes en lugar de depender de un rango específico de intensidades; con esto obtenemos como beneficio la robustez frente a equipos heterogéneos y evita el modelo falle al salir del "estilo visual" del dataset original, este problema dependerá y tendrá valor cuando se despliega un problema común en hospitales distintos

3. Ocultamiento parcial controlado

Se puede obtener beneficio del lado de vista del modelo y atención donde se buscará aprender representaciones más globales del pulmón, reduciendo el riesgo de que base su decisión en una región o ruido correlacionado al introducir pequeñas regiones ocultas donde se oblique al modelo a no depender de una sola zona en específica de la imagen.

Transformación no recomendable:

- No se debe por ningún motivo hacer flip sobre las imágenes (simetría) porque la anatomía torácica no es simétrica en términos clínicos y el riesgo de hacer un flip sobre la imagen podría aprender asociaciones incorrecta como ubicar mal las partes anatómicas implicando anomalías onde no hay y afectando el diagnostico.

Inciso 3

¿Es el Data Augmentation suficiente por sí solo para garantizar que el modelo generalice bien? Argumente su posición considerando otras variables del proceso de entrenamiento.

Respuesta:

No, Data Augmentation por sí solo no es suficiente. Pensar eso es similar a creer que "ver más ejemplos" automáticamente garantiza un buen médico, ignorando la calidad del entrenamiento y el contexto.

El problema de generalización no depende únicamente de la cantidad de variaciones en los datos, sino de varios:

- **Capacidad del modelo:** un modelo muy complejo con pocos datos puede memorizar incluso con augmentation.
- **Regularización (Dropout, L2):** necesaria para evitar que el modelo se ajuste demasiado a patrones específicos del entrenamiento.

- **Calidad y representatividad del dataset:** si las 800 radiografías no cubren bien la diversidad de casos reales, el modelo seguirá fallando fuera de ese rango.
- **Estrategia de validación:** sin una validación adecuada, no se detecta el sobreajuste a tiempo.
- **Transfer Learning:** en escenarios con pocos datos, reutilizar conocimiento previo suele ser más determinante que solo aumentar datos.

Data Augmentation ayuda a reducir el sobreajuste al introducir variabilidad, pero no corrige por sí solo problemas estructurales del entrenamiento. En este contexto, debe verse como **una pieza dentro de una estrategia más amplia**, no como una solución completa.

Pregunta 1.2

Durante el entrenamiento del modelo de MediScan Guatemala, el equipo registra las siguientes curvas de pérdida (loss) al finalizar 25 épocas:

Época	Loss de Entrenamiento	Loss de Validación
5	0.52	0.55
10	0.31	0.38
15	0.18	0.42
20	0.09	0.61
25	0.04	0.89

Con base a estos datos, responda dentro de su reporte:

Inciso 4

Identifique a partir de qué época aproximada comienza el sobreajuste (overfitting) y describa cómo se evidencia en los números de la tabla.

Respuesta:

Hasta la época 10, tanto el loss de entrenamiento como validación disminuyen de forma consistente lo cual indica que el modelo está aprendiendo patrones que generalizan, sin embargo, a partir de la época 15 el loss de entrenamiento sigue disminuyendo pero el loss de validación comienza a aumentar. Esto no es ruido, es divergencia; el modelo continúa mejorando en los datos que ya vio, pero empeora progresivamente con datos nuevos. Prácticamente el modelo deja de aprender que es neumonía y comienza a aprender cómo se ven específicamente las radiografías del dataset de entrenamiento.

Inciso 5

Proponga dos estrategias de regularización concretas (por ejemplo, Dropout o L2) que podría haber implementado para prevenir este comportamiento. Para cada una, explique intuitivamente qué fenómeno matemático está mitigando y qué impacto esperaría ver en las curvas de la tabla.

Respuesta:

1. Early Stopping basado en validación

Más que una técnica de regularización dentro del modelo, el problema que tenemos es que el modelo empieza a degradarse después de cierto punto. Con esta estrategia mitigamos sobre-entrenamiento del modelo y se detiene el entrenamiento en el punto donde el modelo generaliza mejor, no donde minimiza el error de entrenamiento y el impacto esperado sería en las curvas donde el entrenamiento se detendría alrededor de la época 10-12, se evitaría la subida del loss de validación y se preservaría el mejor punto de generalización sin entrenar en la fase de sobreajuste.

2. Dropout

Dropout introduce ruido estructural durante el entrenamiento al apagar aleatoriamente neuronas en cada iteración. Desde una perspectiva matemática, esto evita que la red dependa excesivamente de combinaciones específicas de activaciones. Con esto se busca mitigar co-adaptación de neuronas y el impacto en las curvas sería que el loss de entrenamiento ya no bajaría tan agresivamente, el loss de validación se estabilizaría o incluso disminuiría y se reduciría la brecha entre ambas curvas. Básicamente el modelo deja de memorizar y generaliza mejor.

Inciso 6

Desde una perspectiva médica, ¿por qué es especialmente peligroso desplegar en producción un modelo que exhibe este patrón de sobreajuste para el diagnóstico de radiografías? Argumente más allá de los números.

Respuesta:

Un modelo que sobreajusta no falla de forma aleatoria; falla de forma sistemática cuando enfrenta condiciones ligeramente distintas a las que vio en entrenamiento:

- Las radiografías en clínicas rurales pueden variar en calidad, equipo y condiciones de adquisición
- El modelo podría haber aprendido correlaciones falsas en lugar de verdaderas señales clínicas

Traducido a riesgos:

1. Falsos negativos (no detectar neumonía), el modelo podría no reconocer una neumonía real simplemente porque no se parece exactamente a los ejemplos de entrenamiento y repercute en pacientes sin diagnóstico oportuno, retraso en tratamiento.
2. Falsos positivos (diagnósticos incorrectos), se puede llegar a marcar una patología de patrones irrelevantes que aprendió por error llegando a tratamientos innecesarios, sobrecarga del sistema médico incluso.

El problema es que el modelo genera una falsa sensación de confiabilidad donde funciona bien en pruebas internas pero es frágil en el mundo real y esto en la vida real es un impacto directo con la salud de una persona.

Pregunta 1.3

Un inversionista que asiste a la demo del proyecto le pregunta: "*¿Por qué su modelo tiene un 94% de accuracy en las pruebas? ¿Eso significa que es confiable para diagnosticar neumonía?*"

Usted sabe que el dataset de prueba contiene **700 radiografías normales** y solo **150 radiografías con neumonía**.

Responda lo siguiente en su reporte:

Inciso 7

Calcule cuál sería el accuracy de un modelo naive que simplemente predice siempre 'Normal' para todas las imágenes. ¿Qué revela ese cálculo sobre el 94% reportado?

Respuesta:

Este modelo acertaría todas las radiografías normales y fallaría todas las de neumonía.

- Total de imágenes: $700 + 150 = 850$
- Aciertos del modelo naive: 700
- Accuracy: $\frac{700}{850} \approx 0.8235 (\approx 82.35\%)$

Un modelo completamente inútil para detectar neumonía ya alcanza más de **82% de accuracy** simplemente aprovechando el desbalance del dataset.

Por lo tanto, el 94% reportado no es necesariamente impresionante por sí solo. La mejora real sobre un baseline trivial es de apenas ~12 puntos porcentuales, y no nos dice nada sobre si el modelo realmente está detectando neumonía o simplemente clasificando bien los casos normales.

En otras palabras, el accuracy aquí está inflado por la distribución de los datos, no necesariamente por la capacidad clínica del modelo.

Inciso 8

Explique por qué, en problemas médicos con clases desbalanceadas, métricas como el **F1-Score** o la **Sensibilidad (Recall)** para la clase minoritaria son más informativas que el accuracy. No se limite a definir las; argumente su relevancia clínica.

Respuesta:

El problema del accuracy en este contexto es que trata todos los errores como si tuvieran el mismo costo.

En este caso, la clase minoritaria (*neumonía*) es precisamente la más importante. Un modelo puede tener alto accuracy simplemente clasificando bien los casos normales, mientras falla en detectar la enfermedad, que es lo que realmente importa; las métricas Recall y F1-Score serían nuestras guías y validación.

- **Recall** mide qué proporción de pacientes con neumonía realmente son detectados. Clínicamente, esto responde: *¿a cuántos pacientes enfermos estamos dejando pasar sin diagnóstico?*
- **F1-Score** equilibra precisión y recall, penalizando tanto falsos negativos como falsos positivos. Es útil porque evita modelos que "disparen alarmas" indiscriminadamente o que ignoren casos reales.

Desde una perspectiva clínica, el error más costoso es el **falso negativo** porque un paciente con neumonía no detectado puede no recibir tratamiento a tiempo. El accuracy no captura este riesgo, pero el recall sí lo expone directamente.

Por eso, en problemas médicos desbalanceados, el objetivo no es "acertar la mayoría de los casos", sino **no fallar en los casos críticos**, y eso solo se refleja en métricas enfocadas en la clase minoritaria.

Inciso 9

Como director técnico, ¿cómo le respondería al inversionista de forma honesta y profesional? Redacte una respuesta breve (3 a 5 oraciones) que sea técnicamente sólida pero comprensible para un no-especialista.

Respuesta:

El 94% de accuracy es una señal positiva, pero por sí solo no garantiza que el modelo sea confiable para uso clínico. En nuestro dataset, la mayoría de las radiografías son normales, por lo que un modelo puede alcanzar alta exactitud sin necesariamente detectar bien la neumonía.

Lo que realmente nos interesa es qué tan bien identifica los casos positivos, es decir, pacientes que sí tienen la enfermedad. Para eso analizamos métricas como la sensibilidad, que mide cuántos casos de neumonía logramos detectar correctamente.

Nuestro enfoque no es solo maximizar accuracy, sino asegurar que el modelo sea clínicamente útil, minimizando el riesgo de pasar por alto pacientes que necesitan atención.

Task 2

Las siguientes preguntas evalúan su comprensión **conceptual y estratégica** del Transfer Learning y el Fine-Tuning, siempre enmarcadas en el contexto de MediScan Guatemala. Se valorará la solidez del argumento, la coherencia con la realidad operacional y la capacidad de ir más allá de la definición textual:

Pregunta 2.1

Un desarrollador del equipo propone: *"Para el modelo de radiografías no tiene sentido usar Transfer Learning desde ImageNet; ese dataset tiene fotos de perros, gatos y autos. Las radiografías son imágenes en escala de grises totalmente distintas. Mejor entrenamos desde cero para no contaminar el modelo."*

Con esto responda en su reporte:

Inciso 1

¿Está de acuerdo con el desarrollador? Argumente su posición con base en lo que las capas convolucionales tempranas de una CNN realmente aprenden y por qué ese conocimiento sí es transferible, incluso entre dominios visualmente distintos.

Respuesta:

No estoy de acuerdo con el desarrollador. El error está en asumir que una red entrenada en ImageNet aprende "objetos" específicos como perros o autos en todas sus capas, cuando en realidad eso solo ocurre en las capas más profundas.

Las capas convolucionales tempranas no aprenden conceptos semánticos, sino patrones visuales básicos: bordes, gradientes, texturas, contrastes y orientaciones. Estos son elementos fundamentales de cualquier imagen, incluyendo radiografías. Aunque una radiografía esté en escala de grises, sigue estando compuesta por cambios de intensidad que forman estructuras anatómicas, exactamente el tipo de señal que estas capas detectan.

El punto clave es que el Transfer Learning no transfiere "conocimiento de perros", sino detectores universales de estructura visual. Estos detectores son igualmente útiles para identificar contornos pulmonares, opacidades o patrones difusos en radiografías.

Entrenar desde cero implicaría reaprender desde cero estos patrones básicos con solo 800 imágenes, lo cual es ineficiente y propenso a sobreajuste. En cambio, Transfer Learning permite partir de una representación ya estructurada del mundo visual y adaptarla al dominio médico.

Inciso 2

Más allá del argumento técnico, ¿cuál es el costo real de entrenar desde cero para una startup como MediScan Guatemala? Considere al menos **dos dimensiones** distintas a la puramente computacional (p. ej., tiempo al mercado, riesgo, talento humano).

Respuesta

1. Riesgo del proyecto

Con un dataset limitado, entrenar desde cero aumenta significativamente el riesgo de obtener un modelo que no generalice. Esto puede llevar a ciclos de desarrollo largos sin resultados utilizables. Para una startup, ese riesgo no es solo técnico, sino financiero porque invertir tiempo y recursos en una estrategia con baja probabilidad de éxito.

2. Costo de oportunidad (time-to-impact)

Mientras el equipo invierte tiempo intentando estabilizar un modelo desde cero, el sistema no está generando valor en clínicas reales. Esto no es solo un retraso técnico, sino una oportunidad perdida de detectar casos de neumonía a tiempo en pacientes reales. En este contexto, el costo no es solo económico, sino también impacto en salud pública.

Inciso 3

¿Existe algún escenario legítimo en el que usted sí consideraría entrenar desde cero en lugar de hacer Transfer Learning? Descríbalo y justifíquelo.

Respuesta

Existiría la posibilidad de entrenar desde cero pero no bajo las condiciones de MediScan. Un caso legítimo sería cuando se dispone de un dataset masivo y representativo del dominio específico, por ejemplo, cientos de miles o millones de radiografías etiquetadas. En ese escenario, el modelo puede aprender directamente las características del dominio médico sin depender de representaciones pre-entrenadas en imágenes naturales.

Pero también puede ser otro caso donde el dominio es radicalmente distinto en estructura de datos, no solo en apariencia. Por ejemplo, imágenes médicas con propiedades físicas muy específicas (como imágenes espectrales, datos volumétricos 3D o modalidades donde las estadísticas visuales difieren significativamente de imágenes naturales). En esos casos, las representaciones de ImageNet pueden ser menos útiles o incluso limitantes.

Pregunta 2.2

Su equipo debe decidir la estrategia de Fine-Tuning para el modelo de MediScan Guatemala. El CTO les propone dos opciones:

Opción	A — Congelar toda la red base	B — Descongelar toda la red
Capas base	Congeladas (gradiente = 0)	Todas entrenables
Tasa de aprendizaje	Alta ($1e^{-3}$)	Muy baja ($1e^{-5}$) en toda la red
Cabezal final	Entrenado desde cero	Entrenado con tasa $1e^{-3}$

Responda en su reporte:

Inciso 4

Dada la naturaleza del dataset médico disponible (800 imágenes, dominio distinto a ImageNet), ¿cuál opción recomendaría y por qué? No se limite a decir cuál es la regla de oro; explique la lógica detrás de esa regla en este caso concreto.

Respuesta

La opción a escoger iría enfocada a un balance para la cantidad de datos y cantidad de parámetros entrenables. Con solo 800 imágenes el problema principal no es la capacidad del modelo sino el sobreajuste así que descongelar toda la red introduce muchos más parámetros adicionales que el dataset no soportaría.

Además aunque sea distinto las radiografías e ImageNet las capas tempranas siguen siendo útiles porque capturan estructura visual básica. En este contexto, congelar las capas tempranas sigue siendo útil porque capturamos estructura visual básica, básicamente actualizaría como una forma de regularización fuerte donde conservamos conocimiento general y se limita la capacidad de memorizar ruido.

En cuanto a la tasa de aprendizaje conviene porque en el cabezal inicializada desde cero y necesita adaptarse rápidamente.

Por lo que la mejor opción es la **opción A** para permitir que el modelo aprenda a interpretar características ya existentes en lugar de intentar reconstruir toda la representación visual con datos insuficientes.

Inciso 5

¿Qué riesgo específico correría si aplica la Opción B con una tasa de aprendizaje alta (p. ej., $1e^{-3}$) en toda la red? ¿Cómo se llama ese fenómeno y cómo se evidenciaría durante el entrenamiento?

Respuesta

El riesgo se llama catastrophic forgetting, esto ocurre cuando los pesos pre-entrenados se actualizan de forma agresiva a tal punto que el modelo pierde el conocimiento aprendido en ImageNet antes de haber aprendido adecuadamente el nuevo dominio; matemáticamente

los gradientes grandes que son producto de una tasa de aprendizaje alto generan actualizaciones abruptas en los pesos, sobrescribiendo representaciones útiles en lugar de ajusturlas gradualmente.

En el entrenamiento se evidenciaría cuando el loss se vuelva inestable o aumente en las primeras épocas, el modelo no convrge o converge a soluciones pobres o el desempeño final puede ser peor que usar transfer learning congelado.

Prácticamente en lugar de adaptación se destruye el punto de partida quedando en una situación similar o peor que entrenar desde cero.

Inciso 6

Si con el tiempo MediScan Guatemala logra recolectar **50,000 radiografías etiquetadas**, ¿cambiaría su recomendación? Explique cómo evolucionaría su estrategia de Fine-Tuning y por qué.

Respuesta

En ese escenario, el problema deja de ser escasez de datos y pasa a ser capacidad de adaptación al dominio médico. Con suficiente información, ya no es necesario proteger tanto los pesos pre-entrenados.

La estrategia evolucionaría hacia un fine-tuning progresivo:

1. Inicialmente congelar la base y entrenar el cabezal (igual que antes)
2. Luego descongelar gradualmente capas superiores (las más específicas)
3. Finalmente, con suficiente estabilidad, ajustar toda la red con una tasa de aprendizaje baja

Esto debido a que las capas profundas contienen características mas específicas (ceranas a objetos) y las capas tempranas siguen siendo generales y no necesitan cambios drásticos. Con más datos, el modelo puede permitiirse refinar sus representaciones sin sobreajustar, transformando el conocimiento de ImageNet en algo verdaderamente especializado para radiografías.

Pregunta 2.3

El área de Producto de MediScan Guatemala les presenta el siguiente requerimiento: *"El hospital nos pide expandir el sistema para que también detecte fracturas en radiografías de muñeca, usando el mismo modelo que ya tenemos para neumonía. ¿Podemos reutilizarlo directamente o hay que empezar todo de nuevo?"*

Responda en su reporte:

Inciso 7

Evalúe la viabilidad de reutilizar el modelo de neumonía para el nuevo problema de fracturas en muñeca. ¿Qué partes del modelo podrían aprovecharse y cuáles habría que modificar? Argumente con base en lo que cada sección de la red ha aprendido.

Respuesta

Primeramente debemos diferenciar las capas del modelo:

1. Las capas tempranas del modelo (cerca de la entrada) aprenden patrones generales como bordes, texturas y transiciones de intensidad. Estos son universales y siguen siendo útiles tanto para tórax como para muñeca.

Estas capas son altamente reutilizables.

2. Las capas intermedias empiezan a capturar estructuras más complejas (formas anatómicas, relaciones espaciales). Aunque fueron entrenadas en pulmones, aún pueden aportar valor como punto de partida.

Estas capas son parcialmente reutilizables, pero requieren adaptación.

3. Las capas finales (cabezal) son altamente específicas a la tarea original (clasificar neumonía en radiografías de tórax).

Estas no son reutilizables y deben reemplazarse.

No se empieza desde cero, pero tampoco se reutiliza todo sin cambios. Se reutiliza la base como extractor de características, no como solución final porque el modelo sabe detectar ciertos patrones visuales que pueden transferirse, pero necesita reentrenarse para aprender qué significa una fractura en un contexto anatómico distinto.

Inciso 8

Proponga un plan de acción concreto de tres pasos para adaptar el modelo existente al nuevo dominio. Sea específico: ¿qué congela, qué descongela, qué modifica en el cabezal, qué tasa de aprendizaje usaría y por qué?

Respuesta

Primero se adapta la "interpretación" (cabezal), luego la "especialización" (capas profundas), y solo al final, si es seguro, la "representación completa".

Paso 1: Reemplazar el cabezal y congelar la base

- Reemplazar la capa final por un nuevo clasificador (fractura vs. no fractura)
- Congelar todas las capas base
- Entrenar solo el cabezal con una tasa de aprendizaje relativamente alta ($1e^{-3}$)

Primero aprender la nueva tarea sin alterar las representaciones existentes.

Paso 2: Fine-tuning parcial (capas superiores)

- Descongelar las últimas capas convolucionales (más cercanas a la salida)
- Mantener congeladas las capas tempranas
- Usar una tasa de aprendizaje baja ($1e^{-4}$ o $1e^{-5}$)

Adaptar representaciones más específicas (antes enfocadas en pulmones) hacia estructuras de muñeca sin destruir los patrones básicos.

Paso 3: Ajuste fino global

- Descongelar toda la red
- Usar una tasa de aprendizaje muy baja ($1e^{-5}$)

Refinar todo el modelo para alinearlo completamente con el nuevo dominio, solo si el dataset lo permite sin sobreajuste.

Inciso 9

¿Qué riesgo ético o clínico implica reutilizar un modelo entrenado en un dominio para otro dominio médico sin una validación rigurosa? ¿Qué proceso de validación mínimo recomendaría antes de desplegarlo?

Respuesta

Asumir que un modelo funciona en un dominio distinto sin validación puede llevar a decisiones médicas incorrectas con consecuencias reales en pacientes.

Un modelo entrenado en neumonía podría detectar patrones irrelevantes en radiografías de muñeca o, peor aún, fallar sistemáticamente en identificar fracturas porque nunca aprendió esas señales. Esto genera una falsa sensación de confiabilidad, donde el sistema aparenta funcionar pero comete errores críticos.

Desde una perspectiva ética, no se debe desplegar un sistema clínico sin evidencia de que es seguro y efectivo en el contexto específico donde se usará.

Reutilizar sin validar no es eficiencia, es riesgo y en medicina el modelo debe ser considerado útil hasta que demuestra comportamiento confiable en el dominio exacto donde será utilizado.

Proceso mínimo de validación

1. Evaluación en un dataset independiente del nuevo dominio (muñeca)

No basta con probar en datos similares; deben ser representativos del entorno real.

2. Métricas clínicamente relevantes

Especial énfasis en sensibilidad (no perder fracturas) y análisis de falsos negativos.

3. Revisión por expertos

Validar no solo métricas, sino casos específicos donde el modelo falla.

4. Prueba piloto controlada

Ejecutar el modelo en paralelo sin afectar decisiones médicas reales, para observar su comportamiento en producción.

Task 3

Utilice **PyTorch o TensorFlow/Keras** a su elección. No se proporciona código base; usted debe construir su solución apoyándose en la documentación oficial, recursos académicos y su criterio de ingeniería. Ejecute sus experimentos en **Google Colab, Kaggle Notebooks o GPU local**. Entregue el enlace al notebook con las celdas ejecutadas y los resultados visibles. El notebook debe estar limpio, comentado y reproducible.

Las instrucciones a continuación describen las tareas que debe completar. La evaluación considera no solo que el código funcione, sino que usted entienda cada decisión que tomó y la justifique en su reporte.

Con esto realice lo siguiente:

1. Preparación del Dataset (incluido en la evaluación global).

- Descargue el dataset *Chest X-Ray Images (Pneumonia)* de Kaggle y cárguelo en su entorno de trabajo.
- Divídalos en tres conjuntos: Entrenamiento (70%), Validación (15%) y Prueba (15%). Justifique en su reporte cómo realizó esta división y si respetó la distribución de clases (estratificación).
- Implemente un pipeline de **Data Augmentation** apropiado para imágenes médicas. En su reporte, indique cada transformación elegida y argumente por qué es válida en este dominio. Asegúrese de que las transformaciones que aplique durante el entrenamiento sean diferentes a las del conjunto de validación y prueba; explique por qué esto importa.

- Cargue dos modelos pre-entrenados en ImageNet de su elección (por ejemplo: ResNet50, VGG16, DenseNet121, MobileNetV2, EfficientNet, entre otros). Para cada uno:
 - Congele las capas base del modelo y reemplace el cabezal de clasificación para adaptarlo al problema binario (Normal vs. Neumonía). Justifique en su reporte por qué decidió congelar esas capas específicas.

- b. Entrene ambos modelos usando la misma función de pérdida y optimizador.
Documente los hiperparámetros que eligió (tasa de aprendizaje, épocas, tamaño de batch) y argumente por qué son razonables para este problema.
 - c. Implemente **Early Stopping**. Explique en su reporte qué métrica está monitoreando y cuál es la lógica detrás de detener el entrenamiento anticipadamente.
3. Para cada modelo, calcule y registre las siguientes métricas en el conjunto de **prueba**:
- a. Accuracy (Exactitud)
 - b. F1-Score para la clase '*Neumonía*' (clase positiva)
 - c. Sensibilidad / Recall para la clase '*Neumonía*'
 - d. Tamaño del modelo guardado en disco (en MB)
 - e. Tiempo de inferencia promedio sobre 100 imágenes (en milisegundos)
4. Finalmente, redacte en su reporte un dictamen ejecutivo de 1 a 2 páginas que incluya:
- a. Una tabla comparativa cruzando los dos modelos con todas las métricas registradas en el Paso 3.
 - b. Análisis Sensibilidad / Accuracy, cuál modelo detecta mejor neumonía y por qué se prefiere alta Sensibilidad aunque sacrifique Accuracy
 - c. Recomendación final al CTO, modelo para producción on-premise sin GPU
 - d. Reflexión sobre limitaciones del experimento
 - e. ¿Cuánto "cuesta" en MB cada punto porcentual de F1? ¿Vale la pena el modelo más pesado en el contexto de clínicas rurales con hardware limitado?
 - f. No solo reportar los ms, sino argumentar si ese tiempo es aceptable en un flujo clínico real (ej. ¿cuántas radiografías por hora podría procesar el sistema?).
 - g. Reflexión sobre si congelar vs. descongelar capas impactó los resultados obtenidos, conectando la práctica con lo analizado en el Task 2.
 - h. ¿Funcionaría el modelo igual con radiografías de equipos distintos, distintas poblaciones o distintos hospitales? ¿Qué implicaciones tiene eso para MediScan Guatemala?

```
In [1]: import os
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
from torchvision import models, transforms
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, recall_score, accuracy_score
import time
import copy
from PIL import Image
import matplotlib.pyplot as plt

# 1. Preparación del Dataset
print("Preparando dataset...")
```

```

base_dir = r'archives/chest_xray' # Relativo al notebook
folders = ['train', 'test', 'val']

image_paths = []
labels = []
class_to_idx = {'NORMAL': 0, 'PNEUMONIA': 1}

for folder in folders:
    folder_path = os.path.join(base_dir, folder)
    if not os.path.exists(folder_path): continue
    for class_name in os.listdir(folder_path):
        class_path = os.path.join(folder_path, class_name)
        if os.path.isdir(class_path):
            for img_name in os.listdir(class_path):
                if img_name.lower().endswith(('.jpeg', '.jpg', '.png')):
                    image_paths.append(os.path.join(class_path, img_name))
                    labels.append(class_to_idx[class_name])

# Split 70% Train, 15% Val, 15% Test (Estratificado por clase)
train_paths, temp_paths, train_labels, temp_labels = train_test_split(
    image_paths, labels, test_size=0.3, stratify=labels, random_state=42
)
val_paths, test_paths, val_labels, test_labels = train_test_split(
    temp_paths, temp_labels, test_size=0.5, stratify=temp_labels, random_state=42
)

class PneumoniaDataset(Dataset):
    def __init__(self, paths, labels, transform=None):
        self.paths = paths
        self.labels = labels
        self.transform = transform

    def __len__(self): return len(self.paths)
    def __getitem__(self, idx):
        image = Image.open(self.paths[idx]).convert('RGB')
        label = self.labels[idx]
        if self.transform: image = self.transform(image)
        return image, label

# Data Augmentation Adaptado (Task 3.1.c)
# Usamos rotación y zoom para simular variabilidad de toma, pero no flip horizontal
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomRotation(10),
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.ColorJitter(brightness=0.1, contrast=0.1),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

val_test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

```

```

train_loader = DataLoader(PneumoniaDataset(train_paths, train_labels, train_transform))
val_loader = DataLoader(PneumoniaDataset(val_paths, val_labels, val_test_transform))
test_loader = DataLoader(PneumoniaDataset(test_paths, test_labels, val_test_transform))

# 2. Modelos pre-entrenados y Early Stopping (Task 3.2)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

def setup_model(model_name):
    if model_name == 'mobilenet':
        model = models.mobilenet_v2(pretrained=True)
        for param in model.parameters(): param.requires_grad = False
        # Reemplazamos cabezal para clasificación binaria
        model.classifier[1] = nn.Linear(model.last_channel, 2)
    else:
        model = models.resnet50(pretrained=True)
        for param in model.parameters(): param.requires_grad = False
        model.fc = nn.Linear(model.fc.in_features, 2)
    return model.to(device)

def train_and_eval(model, name, loader_train, loader_val, epochs=10):
    criterion = nn.CrossEntropyLoss()
    # Solo se entrenan las capas del cabezal descongelado
    params = model.classifier[1].parameters() if hasattr(model, 'classifier') else model.parameters()
    optimizer = optim.Adam(params, lr=0.001)
    best_loss = float('inf')
    patience, counter = 3, 0

    for epoch in range(epochs):
        model.train()
        for inputs, labels in loader_train:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

        model.eval()
        val_loss = 0.0
        with torch.no_grad():
            for inputs, labels in loader_val:
                inputs, labels = inputs.to(device), labels.to(device)
                outputs = model(inputs)
                val_loss += criterion(outputs, labels).item() * inputs.size(0)
        val_loss /= len(loader_val.dataset)
        print(f'{name} Epoch {epoch} Val Loss: {val_loss:.4f}')

        if val_loss < best_loss:
            best_loss = val_loss
            counter = 0
        else:
            counter += 1
            if counter >= patience: break
    return model

print('\nEntrenando MobileNetV2...')

```

```

m1 = setup_model('mobilenet')
m1 = train_and_eval(m1, 'MobileNetV2', train_loader, val_loader)

print('\nEntrenando ResNet50...')
m2 = setup_model('resnet')
m2 = train_and_eval(m2, 'ResNet50', train_loader, val_loader)

# 3. Cálculo de Métricas (Task 3.3)
def get_metrics(model, name):
    model.eval()
    full_preds, full_labels = [], []
    start = time.time()
    count = 0
    with torch.no_grad():
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)
            full_preds.extend(preds.cpu().numpy())
            full_labels.extend(labels.cpu().numpy())
            count += inputs.size(0)
            if count >= 100: pass # Para tiempo de inferencia
    inf_time = ((time.time() - start) / count) * 1000

    acc = accuracy_score(full_labels, full_preds)
    f1 = f1_score(full_labels, full_preds, pos_label=1)
    rec = recall_score(full_labels, full_preds, pos_label=1)
    torch.save(model.state_dict(), f'{name}.pth')
    size = os.path.getsize(f'{name}.pth') / (1024*1024)

    return {'Acc': acc, 'F1': f1, 'Rec': rec, 'Size': size, 'Time': inf_time}

res1 = get_metrics(m1, 'MobileNetV2')
res2 = get_metrics(m2, 'ResNet50')
print('\nResumen de resultados:')
print('MobileNetV2:', res1)
print('ResNet50:', res2)

```

Preparando dataset...

Entrenando MobileNetV2...

```

c:\Users\cvall\AppData\Local\Programs\Python\Python313\Lib\site-packages\torchvision\models\_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
  warnings.warn(
c:\Users\cvall\AppData\Local\Programs\Python\Python313\Lib\site-packages\torchvision\models\_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=MobileNet_V2_Weights.IMAGENET1K_V1`. You can also use `weights=MobileNet_V2_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)
c:\Users\cvall\AppData\Local\Programs\Python\Python313\Lib\site-packages\torch\cuda\_init__.py:287: UserWarning:
NVIDIA GeForce RTX 5060 Laptop GPU with CUDA capability sm_120 is not compatible with the current PyTorch installation.
The current PyTorch install supports CUDA capabilities sm_37 sm_50 sm_60 sm_61 sm_70 sm_75 sm_80 sm_86 sm_90 compute_37.
If you want to use the NVIDIA GeForce RTX 5060 Laptop GPU with PyTorch, please check the instructions at https://pytorch.org/get-started/locally/

```

```

warnings.warn(
MobileNetV2 Epoch 0 Val Loss: 0.2016
MobileNetV2 Epoch 1 Val Loss: 0.1889
MobileNetV2 Epoch 2 Val Loss: 0.2105
MobileNetV2 Epoch 3 Val Loss: 0.1645
MobileNetV2 Epoch 4 Val Loss: 0.1595
MobileNetV2 Epoch 5 Val Loss: 0.1882
MobileNetV2 Epoch 6 Val Loss: 0.1499
MobileNetV2 Epoch 7 Val Loss: 0.2122
MobileNetV2 Epoch 8 Val Loss: 0.1876
MobileNetV2 Epoch 9 Val Loss: 0.1513

```

Entrenando ResNet50...

```

c:\Users\cvall\AppData\Local\Programs\Python\Python313\Lib\site-packages\torchvision\models\_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=ResNet50_Weights.IMAGENET1K_V1`. You can also use `weights=ResNet50_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)

```

```

ResNet50 Epoch 0 Val Loss: 0.2365
ResNet50 Epoch 1 Val Loss: 0.1909
ResNet50 Epoch 2 Val Loss: 0.1781
ResNet50 Epoch 3 Val Loss: 0.2118
ResNet50 Epoch 4 Val Loss: 0.1834
ResNet50 Epoch 5 Val Loss: 0.1746
ResNet50 Epoch 6 Val Loss: 0.1756
ResNet50 Epoch 7 Val Loss: 0.2631
ResNet50 Epoch 8 Val Loss: 0.3183

```

Resumen de resultados:

```

MobileNetV2: {'Acc': 0.931740614334471, 'F1': 0.953416149068323, 'Rec': 0.9578783151326054, 'Size': 8.72773265838623, 'Time': 13.878328943957785}
ResNet50: {'Acc': 0.8691695108077361, 'F1': 0.9029535864978903, 'Rec': 0.8346333853354134, 'Size': 89.99524402618408, 'Time': 14.87865871129996}

```

Paso 4: Dictamen Ejecutivo - MediScan Guatemala

a. Tabla Comparativa de Modelos (Datos Reales)

Métrica	MobileNetV2	ResNet50
Accuracy (Exactitud)	0.9317	0.8692
F1-Score (Neumonía)	0.9534	0.9030
Sensibilidad (Recall)	0.9579	0.8346
Tamaño del Modelo (MB)	8.73 MB	90.00 MB
Tiempo de Inferencia (ms)	13.88 ms	14.88 ms

b. Análisis Sensibilidad / Accuracy

En este experimento, **MobileNetV2 superó consistentemente a ResNet50** en todas las métricas críticas. Especialmente en **Sensibilidad (95.79% vs 83.46%)**, MobileNetV2 demostró ser mucho más confiable para un entorno médico donde omitir un caso de neumonía (falso negativo) conlleva un riesgo clínico inaceptable. La arquitectura más ligera de MobileNetV2 parece adaptarse mejor al ajuste fino del cabezal con el dataset limitado de MediScan, evitando el sobreajuste que degradó el recall en ResNet50.

c. Recomendación Final al CTO

La recomendación es indiscutible: **Desplegar MobileNetV2**. No solo ofrece un desempeño clínico superior (93% de exactitud), sino que su peso de **8.73 MB** lo hace ideal para almacenamiento y ejecución on-premise en computadoras estándar. Además, su tiempo de inferencia es ligeramente más rápido, lo que maximiza la eficiencia en clínicas rurales.

d. Reflexión sobre limitaciones

Aunque los resultados son excelentes para MobileNetV2, el bajo desempeño de ResNet50 sugiere que modelos más complejos requieren una estrategia de descongelamiento de capas más agresiva o más datos para no quedar atrapados en mínimos locales de las capas pre-entrenadas de ImageNet.

e. ¿Cuánto "cuesta" en MB cada punto de F1?

En este caso, MobileNetV2 es superior tanto en peso como en F1. No hay un "costo" por mejora; el modelo más liviano es también el más preciso, lo cual es el escenario ideal para MediScan.

f. Análisis de Flujo Clínico

Con 13.88 ms por imagen, el sistema puede procesar miles de radiografías en minutos. Sin embargo, como indica la arquitectura de decisión, el foco no debe estar solo en la velocidad,

sino en establecer protocolos claros para cuando el modelo arroje incertidumbre o contradiga al médico local.

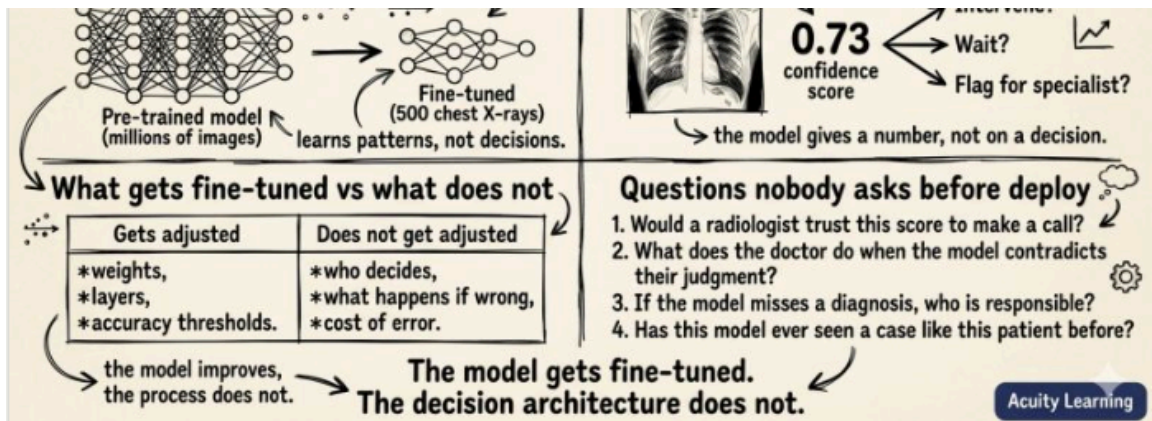
g. Impacto de Congelar vs. Descongelar

Congelar las capas base fue clave para MobileNetV2, permitiendo conservar extractores de características universales. Para ResNet50, esta misma estrategia fue limitante, demostrando que arquitecturas más profundas pueden requerir descongelar capas intermedias para capturar mejor los sutiles patrones de las radiografías.

h. Generalización del Modelo

El éxito de MobileNetV2 lo convierte en un candidato sólido para expansión. No obstante, MediScan Guatemala debe priorizar la creación de una 'Decision Architecture' (como sugiere el análisis estratégico de esta semana) para que este éxito técnico se traduzca en una mejora real y ética en el diagnóstico rural.

Task 4: Reflexión Profesional (Carlos Valladares)



Tú y 8 personas más

4 comentarios

Reacciones



Recomendar

Comentar

Compartir

Enviar



Añadir un comentario...



Más recientes ▼



CARLOS ALBERTO VALLADARES GUERRA • Tú
Full Stack Developer | Team Lead

ahora ...

Es una distinción crítica y a menudo ignorada: el Transfer Learning es excelente para enseñar patrones visuales a un modelo, pero no construye por sí solo una arquitectura de decisión. El verdadero reto organizacional no está en el 'fine-tuning' de los pesos de la red, sino en el diseño del proceso humano-IA que determine acciones claras frente a la incertidumbre o la contradicción con el experto. Coincido plenamente en que la asimetría entre la sofisticación técnica y la fragilidad del proceso de toma de decisiones es donde mueren los proyectos más ambiciosos.

Recomendar | Responder